Meeting 6 SP23

Pipeline Gating

@nebu

Outline

Overview of Branch Prediction

Reducing Energy: Pipeline Gating

Benchmarks/Results

Section 1

Overview of Branch Prediction

Pipelining Requires *Prediction*

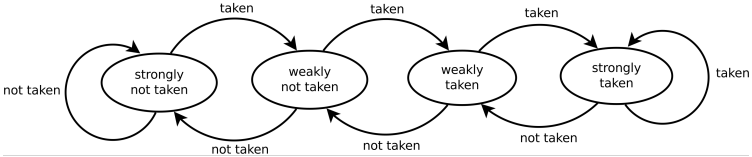
- Imagine a conditional branch instruction enters the pipeline.
- Until we figure out whether to branch (this requires going through a couple of pipeline stages…) what do we do?
- Need to "predict" which way the branch will go to figure out where to issue next instructions from.

Ways to Predict

- "Static": always predict branches are taken (or untaken). Or some other static scheme, like always take backward branches, never forward branches, etc. etc.
- "Random": flip a coin to decide whether to take the branch. Makes timing nondeterministic.
- "Dynamic": much better, learns with the program at runtime.

Dynamic Methods: One Level Saturating Counters

- One level, one bit: record the last outcome of the branch, assume the next one will go the same way.
- One level, two bit: is this FSM:



- Better since *two* branches that go the other way are required to change state.
- There's a table of predictions indexed by the address of the instruction for predicting each branch.

Dynamic Methods: Two Level

- Key idea is to catch repeating patterns of branches, for example: `100100100100`…
- Trick is to:
  - Remember the past n directions of the branch.
  - Have 2^n two bit saturating counters, one for each possible history pattern (it'll be in $\{0, 1\}^n$).

Local vs. Global Prediction

- Local: maintain a history for each individual branch instruction.
- Global: use a *global* history table instead of one local to each branch.
  - If branch directions are correlated with each other, performs better, else no.
- Global could use a two-level predictor, but is only effective for large n, requiring lots of hardware (since it scales as 2^n).

gshare/gselect

- Can mix the branch PC and the global history in various ways to get a prediction.
- is `gshare` if it XOR's them.
- is `gelect` if it concat's them.

Hybridization (McFarling predictors)

- Have multiple prediction mechanisms.
- Pick the predictor that's made the best predictions in the past. (meta-prediction)
- Or use a majority function on predictor outputs.

Misprediction has a Power Cost

- Lots of speculation means you'll be wrong sometimes.
- Being wrong has a cost: you've done useless work, parts of the pipeline need to be flushed and "restarted" with the correct execution path.

Quantification of Extra Work

- Let extra work be:

$$E_w = \frac{\texttt{seeninstr} - \texttt{committedinstr}}{\texttt{committedinstr}}$$

- For a non-speculative processor, $E_w = 0$ (`committedinstr == seeninstr`).
- Partially executed instructions aren't committed if the branch was mispredicted, so the processor spent time doing useless work.

Section 2

Reducing Energy: Pipeline Gating

The Paper

S.Manne, A. Klauser and D. Grunwald, “Pipeline gating: speculation control for energy reduction,” *Proceedings. 25th Annual International Symposium on Computer Architecture (Cat. No.98CB36235)*, Barcelona, Spain, 1998, pp. 132-141, doi: 10.1109/ISCA.1998.694769.

Pipeline Gating

- Power can be reduced in electronic components by *gating* them, which reduces the overall leakage power of the circuit.
- Pipeline gating: prevent incorrectly speculated instructions from entering the pipeline in the first place to reduce power used by at least the initial pipeline stages.

Potential Pitfalls

- We need *confidence estimators* to figure out how sure we are that we're predicting the branch correctly.
- If the confidence estimate is off, we'll hurt performance by gating the pipeline when we shouldn't be.

Energy != Power

- Recall:

$$\texttt{energy} = \texttt{power} \times \texttt{time}$$

- If we take more time to do something (i.e., impact performance), even if we save some power the total energy may be *negatively* impacted.

Key Idea

Save energy without sacrificing performance by using accurate confidence estimators.

Assuming A Good Confidence Estimator...

- This paper uses number of unresolved low-confidence branches to decide when to gate, then gates the fetch stage.[1]
- No more than (say) one low-confidence branch's instructions are issued.

[1] The decode stage can be gated too, wait for benchmarks...

Confidence Estimation Metrics

- SPEC (specificity):

$$\frac{\#\texttt{detected low confidence}}{\#\texttt{actually mispredicted}}$$

- PVN (predictive value of negative test):

$$P(\texttt{low confidence will be incorrectly predicted})$$

Confidence Estimation is a Tradeoff b/w SPEC and PVN

- Mark *all* branches as “low confidence”:
  - SPEC is perfect: we detected every misprediction!
  - PVN is just the mispredict rate. Not useful.
- Need a tradeoff.

Back of the Envelope Calculation

- Assume average PVN=30%.
- Let there be N low confidence branches in pipeline. Then, probability that at least one of them is mispredicted is

$$1 - (1 - \texttt{PVN})^N \approx 0.51$$

  for $N = 2$.
- 51% chance that one of the two branches was mispredicted – gate!
- N is called the *gating threshold*.

Section 3

Benchmarks/Results

Methodology

- Used SPECint (95). No SPECfp. The processor's parameters are:

Parameter	Configuration
L1 Icache	256:64:2 (32 kB) - 1 cycle*
L1 Dcache	256:32:2 (16 kB) - 1 cycle
L2 Combined Cache	512:64:2 (64 kB) - 6 cycles
Memory	128 bit wide - 18 + 2 X chunks cycles
Branch Pred. (McFarling)	2k gshare + 2k bimodal + 2k meta
Branch Pred. (Gshare)	8k gshare entries
BTB	1024 entry, 4-way set associative
Return Address Stack	32 entry queue
ITLB	64 entry, fully associative
DTLB	128 entry, fully associative
Ifetch Queue	8 instructions

Branch Predictors Used

- `gshare`
- McFarling: uses a meta-predictor to choose between `gshare` and usual bimodal branch predictor.
- Branch misprediction penalty is seven cycles if instruction is in L1 i-cache.

Programs

- Compiled a bunch of applications with `gcc` with various standard misprediction rates on each predictor:

			McFarling			Gshare		
Name	Comm Inst	Inst/ Branch	Exec Cycles	Fetch Inst	MisPred Rate (%)	Exec Cycles	Fetch Inst	MisPred Rate (%)
compress	80.4	5.6	44.7	120.9	9.9	44.6	119.8	9.8
gcc	250.9	5.0	249.3	413.3	12.2	282.0	461.3	21.4
go	548.1	6.8	508.8	1043.2	23.9	544.1	1127.3	32.2
ijpeg	252.0	12.6	112.7	316.6	10.4	114.4	320.3	12.2
li	183.3	4.4	100.3	275.2	6.9	106.6	286.3	9.4
m88ksim	416.5	4.6	282.5	544.9	4.6	290.1	555.6	6.5
perl	227.5	5.2	276.8	361.3	11.3	305.0	403.4	21.3
vortex	180.9	6.2	119.7	192.8	1.7	127.7	211.1	5.0

Confidence Estimators: Saturating Counters.

- Use the predictor's internal state.
- For McFarling, "Both Strong" means both its internal predictors have the same (strong) prediction.
- For gshare, "distance" uses the idea that mispredictions are clustered.
  - The conditional probability that a branch within d instructions of a mispredicted branch is higher for small d.

Confidence Estimators: JRS

- Use a saturating-resetting miss distance counter (MDC) to keep track of past prediction correctness.
- Correct prediction increments MDC, incorrect resets it.
- Confidence reached when MDC is above a threshold.

Confidence Estimators: Static For Benchmarking

- For benchmarking, run the entire program, note down branches that were often mispredicted, and mark them as low confidence.
- Sets a “gold standard” for confidence estimation based on threshold.

One Last One for Benchmarking

- “Perfect” pipeline stalling, to see how much of the possible performance we’re able to get out of estimation.
- Sets a “perfect standard” for confidence estimation.

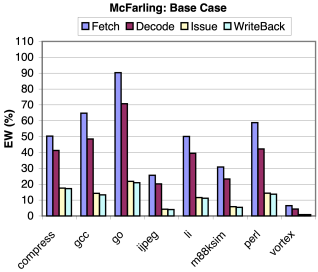
Quantifying our Confidence Estimators

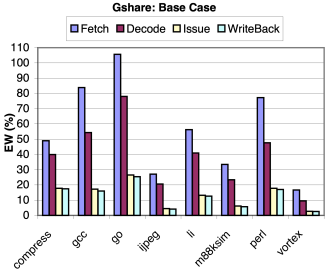
- Use SPEC and PVN!

Gshare		
Conf Pred	SPEC	PVN
static	87.5	27.5
JRS	72.8	37.1
distance=4	71.9	25.8
McFarling		
Conf Pred	SPEC	PVN
static	88.4	26.3
JRS	65.9	30.7
Both Strong	77.2	20.3

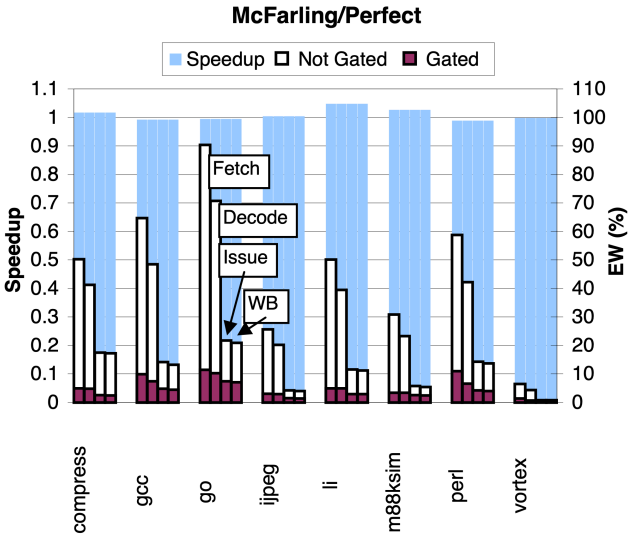
Test!

- First, the E_w (extra work) done due to mispredicted branches by the predictors without any pipeline gating.

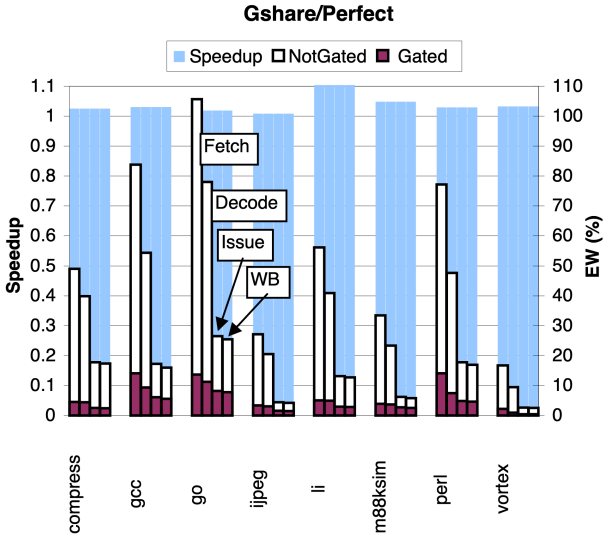




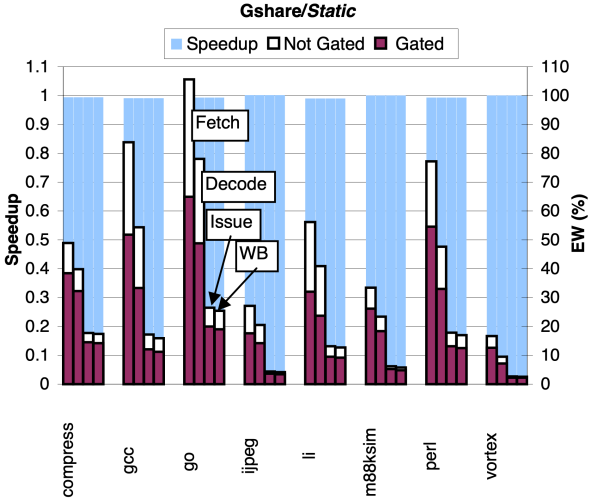
With Confidence Estimators



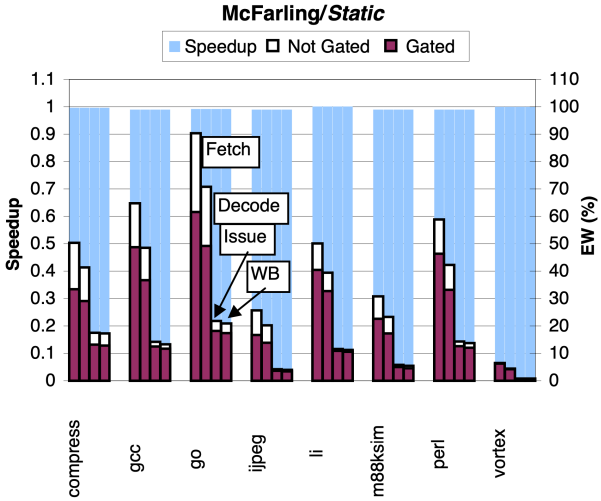
With Confidence Estimators



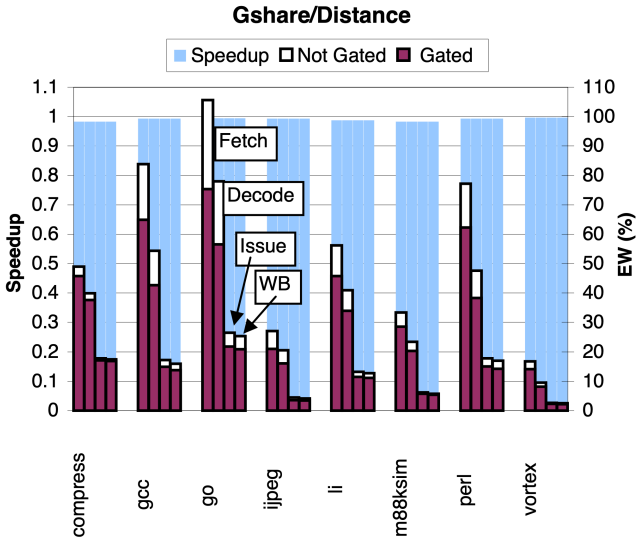
With Confidence Estimators



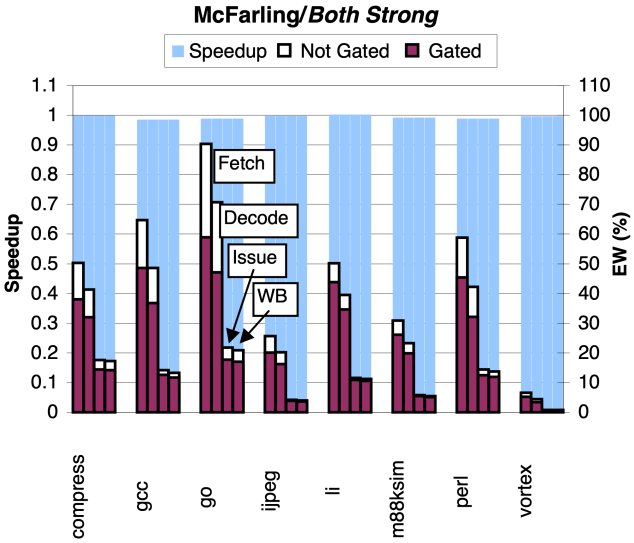
With Confidence Estimators



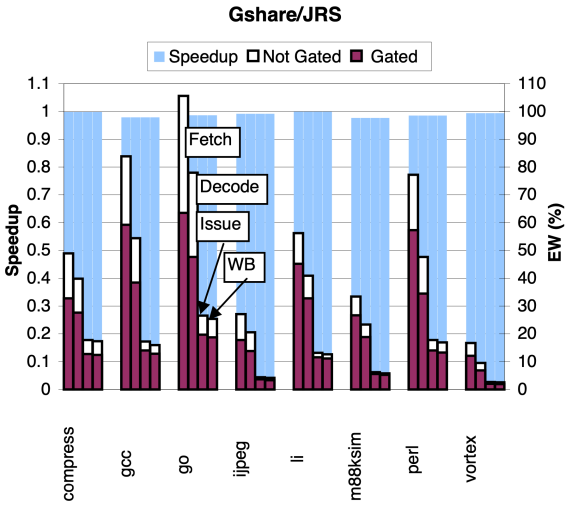
With Confidence Estimators



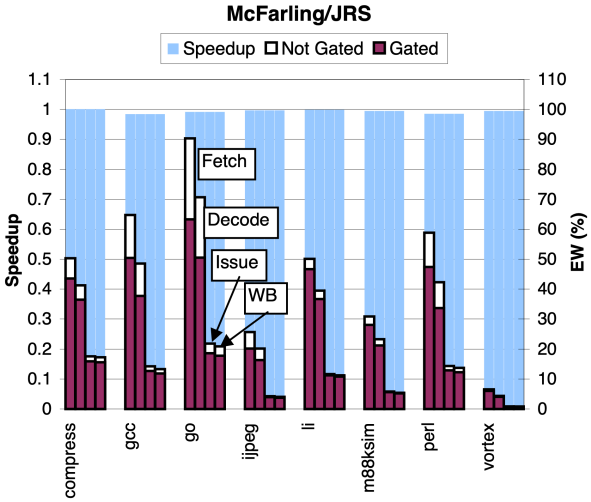
With Confidence Estimators



With Confidence Estimators

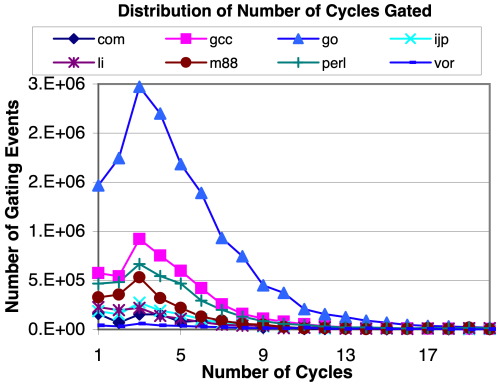


With Confidence Estimators



Actual Stall Cycles

- Gating the pipeline only stalls the processor for ~2-4 cycles.



Other Benchmarks (See Paper for Details)

- Varying JRS thresholds we mentioned – see paper.
- Varying gating threshold N: they found the best value was $N = 2$.
- Varying pipeline structure, gating at decode: didn't make a huge difference since gating isn't for enough cycles to offset the cost of letting bad fetches happen.